

PROJECT REPORT 3 ON

DATA AUTOMATION

Featuring

The Prioritizer: a tool to identify high-priority ecological restoration sites

TABLE OF CONTENTS

I. Introduction

II. Inputs and Outputs

III. Reviewing the Script

- Overview
- 1. User-Defined Inputs and Setup
- 2. Calculate Feature Overlap
- 3. Quantify Proximity to Objects
- 4. Value Intersection Zones
- 5. Isolate Sites with Maximum Value
- Input & Output Maps

IV. Testing Generalizability

- The Scenario
- What's New?
- Inputs Map
- Output Map & Conclusion

I. INTRODUCTION

The Prioritizer is a generalized geospatial software tool, in the form of a python script, optimized to identify high-priority ecological restoration sites within any user-defined area, provided sufficient data is available. It prioritizes sites based on:

1. Proximity to sources of ecological degradation (e.g., trails, campgrounds, roads, toxic waste dumps), which influence the level of ecological impact and restoration urgency.
2. The number of species utilizing a given area, reflecting its ecological significance and conservation value.

More generally, however, this tool can be viewed as a flexible framework for prioritizing spatial areas based on proximity to defined objects of interest and overlap with valued features, making it adaptable for a wide range of applications. For example, instead of being used to identify high-priority ecological restoration sites, The Prioritizer could be used to identify sites with high exposure to natural disaster risk. In this example, valued features could be polygons representing hazard zones (floodplains, landslide-susceptible areas, and wildfire-prone areas) and defined objects could be points or polygons representing dense human population centers or critical infrastructure.

The following section, **II. Inputs and Outputs**, describes these features of the tool in both the context for which it is optimized—ecological restoration—and the alternate context of natural disaster risk.

Sections **III. Reviewing the Script** and **IV. Testing Generalizability** stick with the ecological restoration context to explain how the tool works and to prove its generalizability. Section III relies on the case presented in my Data Interpretation report to review the script, and figures are recycled from that report when appropriate to show how the code works. Section IV works through a similar but distinct hypothetical scenario involving ecological restoration site prioritization in a different region, with different species and sources of degradation.

II. INPUTS AND OUTPUTS

The Prioritizer requires users to input their custom data and settings at the beginning, after which the script runs automatically to completion, requiring no further interaction to produce its output feature class.

Inputs

- Local Base Directory and Geodatabase Paths
 - The script is designed to run locally, enabling users to define a base directory and organize their data into separate geodatabases for inputs and outputs.
- Valued Features
 - In the ecological restoration context, valued features are feature classes representing critical habitat polygons for species listed under the Endangered Species Act (ESA). These features are used to prioritize restoration sites based on ecological significance and conservation value.
 - In the natural disaster risk analysis context, valued features could represent polygons describing exposure to hazards like wildfires, floods, or landslides.
- Defined Objects of Interest
 - In the ecological restoration context, objects of interest are spatial features contributing to ecological degradation in protected areas, such as trails, backcountry campgrounds, roads, or pit toilets.
 - In the natural disaster risk analysis context, defined objects of interest could be disaster-vulnerable entities such as human population centers or critical infrastructure.

Outputs

- The final output of this script is a single feature class containing only the highest-valued polygons based on the user's criteria. These polygons are selected according to their proximity to defined objects of interest and overlap with valued features
- In the ecological restoration context, polygons in the output feature class represent the potential ecological restoration sites with the highest calculated restoration value, based on the number of overlapping ESA-listed species and proximity to ecological degradation sources.
- In the natural disaster risk analysis context, output polygons could represent areas with the highest calculated risk exposure, based on the number of overlapping hazard zones (floodplains, landslide-susceptible areas, wildfire-prone areas) and their proximity to disaster-vulnerable entities.

III. REVIEWING THE SCRIPT - OVERVIEW

1. User-Defined Inputs and Setup

```
# user-defined base directory and geodatabase paths
base_dir = r"C:\Modeling Geographical Objects\Data_Automation\Data_Automation_Report" # base
input_gdb = rf"{base_dir}\Input.gdb" # input gdb
output_gdb = rf"{base_dir}\Data_Automation.gdb" # output gdb
```

```
# List of input species feature classes
species_list = [
    rf"{input_gdb}\VoToad",
    rf"{input_gdb}\SMYLE",
    rf"{input_gdb}\Bighorn",
    rf"{input_gdb}\MYLF"
]
```

```
# corresponding common names for each species
common_names = ["Species1", "Species2", "Species3", "Species4"]
```

```
# ecological degradation source feature class
source_of_degradation = rf"{input_gdb}\trails_clipped"
```

```
# buffer distances (meters)
buffer_distances = [500, 1500, 3000]
```

```
# buffer values
buffer_values = {500: 5, 1500: 3, 3000: 1}
```

2. Calculate Feature Overlap

```
# dictionary to store paths for dissolved feature classes
dissolved_fc_paths = {}
```

```
# Loop through zipped feature classes/common names lists
for fc, common_name in zip(feature_classes, common_names):
    dissolved_fc = rf"{output_gdb}\{common_name}_dissolved"
    arcpy.management.Dissolve(fc, dissolved_fc)
    arcpy.management.AddField(dissolved_fc, f"value_{common_name}", "SHORT")
    arcpy.management.CalculateField(dissolved_fc, f"value_{common_name}", "1")
    dissolved_fc_paths[common_name] = dissolved_fc
```

```
# pull in file paths from the dictionary
in_features = list(dissolved_fc_paths.values())
out_union = rf"{output_gdb}\CriticalHabitat_Union"
arcpy.analysis.Union(in_features, out_union)
```

```
# add new field to the union fc
new_field = "Overlap_Value"
arcpy.management.AddField(out_union, new_field, "SHORT")
```

```
# construct dynamic summation expression
# creates a list of field names based on the common names list
value_fields = [f"!value_{name}!" for name in common_names]
# joins field names with " + " to create summation expression
expression = " + ".join(value_fields)
```

```
# Calculate the Value_Sum field
arcpy.management.CalculateField(out_union, new_field, expression, "PYTHON3")
```

3. Quantify Proximity to Objects

```
# Dissolve the degradation source feature class
dissolved_degradation = rf"{output_gdb}\Dissolved_Degradation"
arcpy.management.Dissolve(
    in_features=source_of_degradation,
    out_feature_class=dissolved_degradation
)
```

```
# generate initial buffers
buffered_layers = []
for distance in buffer_distances:
    buffer_output = rf"{output_gdb}\Source_Buffer_{distance}m"
    arcpy.analysis.Buffer(
        in_features=dissolved_degradation,
        out_feature_class=buffer_output,
        buffer_distance_or_field=f"{distance} Meters",
        dissolve_option="NONE"
    )
    buffered_layers.append(buffer_output)
```

```
# assign values to buffers
for buffer_output, distance in zip(buffered_layers, buffer_distances):
    arcpy.management.AddField(buffer_output, "BufferValue", "SHORT")
    buffer_value = buffer_values[distance]
    arcpy.management.CalculateField(buffer_output, "BufferValue", buffer_value, "PYTHON3")
```

```
donut_buffers = []
for i in range(len(buffer_distances) - 1, 0, -1):
    outer_buffer = buffered_layers[i]
    inner_buffer = buffered_layers[i - 1]
    donut_output = rf"{output_gdb}\Donut_Buffer_{buffer_distances[i]}m"
    arcpy.analysis.Erase(
        in_features=outer_buffer,
        erase_features=inner_buffer,
        out_feature_class=donut_output
    )
    donut_buffers.append(donut_output)

donut_buffers.append(buffered_layers[0])
```

```
# Path for merged output
merged_buffers = rf"{output_gdb}\Merged_Donut_Buffers"
```

```
# Combine all donut buffers
arcpy.management.Merge(
    inputs=donut_buffers,
    output=merged_buffers
)
```

4. Value Intersection Zones

```
# output
restoration_with_habitat = rf"{output_gdb}\Restoration_With_Habitat"

# intersect
arcpy.analysis.Intersect(
    in_features=[out_union, merged_buffers],
    out_feature_class=restoration_with_habitat,
    join_attributes="ALL"
)
```

```
# add field for Restoration Score
arcpy.management.AddField(restoration_with_habitat, "RestorationScore", "FLOAT")

# calculate field as buffer value * habitat overlap value
restoration_expression = "!BufferValue! * !Overlap_Value!"
arcpy.management.CalculateField(
    in_table=restoration_with_habitat,
    field="RestorationScore",
    expression=restoration_expression,
    expression_type="PYTHON3"
)
```

5. Isolate Sites with Maximum Value

```
# Make a feature layer for the restoration data
arcpy.management.MakeFeatureLayer(
    in_features=restoration_with_habitat,
    out_layer="RestorationLayer"
)

# Select polygons with the maximum RestorationScore
arcpy.management.SelectLayerByAttribute(
    in_layer_or_view="RestorationLayer",
    selection_type="NEW_SELECTION",
    where_clause="RestorationScore = (SELECT MAX(RestorationScore) FROM Restoration_With_Habitat)"
)

# Define the output feature class for high-priority sites
highest_priority_sites = rf"{output_gdb}\Highest_Priority_Sites"

# Copy the selected features
arcpy.management.CopyFeatures(
    in_features="RestorationLayer",
    out_feature_class=highest_priority_sites
)

# Define the output for the dissolved feature class
dissolved_priority_sites = rf"{output_gdb}\Dissolved_Highest_Priority_Sites"

# Dissolve contiguous polygons by RestorationScore
arcpy.management.Dissolve(
    in_features=highest_priority_sites,
    out_feature_class=dissolved_priority_sites,
    dissolve_field="RestorationScore",
    multi_part="SINGLE_PART"
)
```

III. Reviewing the Script

I. User-Defined Inputs and Setup

I.1 Base Directory and Geodatabase paths

I.2 Target Species

I.3 Ecological Degradation Sources

III. Reviewing the Script

I. User-Defined Inputs and Setup

I.1 Base Directory and Geodatabase paths

I.2 Target Species

I.3 Ecological Degradation Sources

Please refer to section **II. Inputs and Outputs** for further discussion of this part of the script.

1.1 Base directory and GDB paths

```
# user-defined base directory and geodatabase paths
base_dir = r"C:\Modeling Geographical Objects\Data_Automation\Data_Automation_Report" # base directory
input_gdb = rf"{base_dir}\Input.gdb" # input GDB
output_gdb = rf"{base_dir}\Data_Automation.gdb" # output GDB
```

1.2 Target Features

Users should replace these feature classes with their own. Here, each contains the critical habitat polygons for a different ESA-listed species. There is no limit to how many target features can be included in this analysis. Users can replace common names with whatever they desire, perhaps the common names of the target species feature class with which they correspond.

```
# list of input species feature classes
species_list = [
    rf"{input_gdb}\YoToad",
    rf"{input_gdb}\SNYLF",
    rf"{input_gdb}\Bighorn",
    rf"{input_gdb}\MYLF"
]

# corresponding common names for each species
common_names = ["Species1", "Species2", "Species3", "Species4"]
```

1.3 Objects of Interest

Here, any point source of ecological degradation will suffice (e.g., trails, backcountry campgrounds, roads, cities, mines, waste dumps, etc.). Degradation sources are buffered and valued to quantify their impact zones.

Users should:

- input their object feature class
- adjust buffer distances as desired
- value buffer distances as desired

```
# ecological degradation source feature class
source_of_degradation = rf"{input_gdb}\trails_clipped"

# buffer distances (meters)
buffer_distances = [500, 1500, 3000]

# buffer values
buffer_values = {500: 5, 1500: 3, 3000: 1}
```

III. Reviewing the Script

2. Calculate Feature Overlap

2.1 Dissolve, Add Field, and Calculate Field

2.2 Union

2.3 Calculate Overlap Values

III. Reviewing the Script

2. Calculate Feature Overlap

2.1 Dissolve, Add Field, and Calculate Field

2.2 Union

2.3 Calculate Overlap Values

Dissolved feature classes for each of four endangered species, with an added value field populated by a constant, 1.

Species #1

	OBJECTID *	Shape *	Shape_Length	Shape_Area	value_MYLF
1	1	Polygon	407628.698164	893422364.108904	1

Species #2

	OBJECTID *	Shape *	Shape_Length	Shape_Area	value_Bighorn
1	1	Polygon	1406084.316992	1690350307.529212	1

Species #3

	OBJECTID *	Shape *	Shape_Length	Shape_Area	value_SNYLF
1	1	Polygon	1911532.515201	4406419313.13419	1

Species #4

	OBJECTID *	Shape *	Shape_Length	Shape_Area	value_YoToad
1	1	Polygon	1218336.492239	3032910418.219614	1

2.1 Dissolve, Add Field, and Calculate Field

This step simplifies the input species data by dissolving each feature class into a single feature, creating one feature per species. A loop is used to iterate over two parallel lists: one containing the feature classes and another containing their corresponding common names. For each pair, the following steps are performed:

1. Dynamically define the output feature class path
2. **Dissolve**
3. Add a value field and set it to 1
4. Store the output path in a dictionary for easy access in the next step

```
: # dictionary to store paths for dissolved feature classes
dissolved_fc_paths = {}

# loop through zipped feature classes/common names lists
for fc, common_name in zip(feature_classes, common_names):
    dissolved_fc = rf"{output_gdb}\{common_name}_dissolved"
    arcpy.management.Dissolve(fc, dissolved_fc)
    arcpy.management.AddField(dissolved_fc, f"value_{common_name}", "SHORT")
    arcpy.management.CalculateField(dissolved_fc, f"value_{common_name}", "1")
    dissolved_fc_paths[common_name] = dissolved_fc
```

III. Reviewing the Script

2. Calculate Feature Overlap

2.1 Dissolve, Add Field, and Calculate Field

2.2 Union

2.3 Calculate Overlap Values

Post-Union Attribute Table (subset)

	Shape *	value_YoToad	value_Bighorn	value_MYLF	value_SNYLF	Shape_Length	Shape_Area
1	Polygon	0	1	1	1	2895.676041	1789.365528
2	Polygon	0	0	1	1	16143.142646	18922.986547
3	Polygon	0	1	0	1	298377.430804	109007101.549606
4	Polygon	1	1	0	1	300725.829517	112928314.292886
5	Polygon	1	1	0	0	291097.333257	158857687.035718
6	Polygon	0	1	1	0	310482.407795	218261454.732245
7	Polygon	1	0	0	1	577142.339493	604465740.706475
8	Polygon	0	0	1	0	460820.846986	636666730.578154
9	Polygon	0	1	0	0	001982.655186	047893833.228922
10	Polygon	0	0	0	1	928129.719516	1272275697.10915
11	Polygon	1	0	0	0	943850.774675	1681421201.78774

Note that each post-union polygon retains the individual species value fields from its input features.

Attribute Table Subset With Summed Value Field (subset)

	Shape *	value_YoToad	value_Bighorn	value_MYLF	value_SNYLF	Value_Sum	Shape_Length	Shape_Area
1	Polygon	1	1	0	1	3	300725.829517	112928314.292886
2	Polygon	0	1	1	1	3	2895.676041	1789.365528
3	Polygon	1	1	0	0	2	291097.333257	158857687.035718
4	Polygon	1	0	0	1	2	577142.339493	604465740.706475
5	Polygon	0	1	1	0	2	310482.407795	218261454.732245
6	Polygon	0	1	0	1	2	298377.430804	109007101.549606
7	Polygon	0	0	1	1	2	16143.142646	18922.986547
8	Polygon	1	0	0	0	1	943850.774675	1681421201.78774
9	Polygon	0	1	0	0	1	001982.655186	047893833.228922
10	Polygon	0	0	1	0	1	460820.846986	636666730.578154
11	Polygon	0	0	0	1	1	928129.719516	1272275697.10915

Note the new Value_Sum field, representing the number of endangered species that use each polygon as critical habitat.

2.2 Union

Combines the dissolved species habitats into a single feature class, capturing all overlapping areas.

```
: # pull in file paths from the dictionary
in_features = list(dissolved_fc_paths.values())
out_union = rf"{output_gdb}\Critical_Habitat_Union"
arcpy.analysis.Union(in_features, out_union)
```

2.3 Calculate Overlap Values

This section calculates a combined overlap score for each feature in the union feature class by summing the values assigned to individual species fields (created during the dissolve step). The score represents the total number of species overlapping in a given area.

1. Add new field for overlap scores
2. Construct a dynamic expression to sum value fields
3. Calculate overlap score

```
: # add new field to the union fc
new_field = "Overlap_Value"
arcpy.management.AddField(out_union, new_field, "SHORT")

# construct dynamic summation expression
# creates a list of field names based on the common names list
value_fields = [f"!value_{name}!" for name in common_names]
# joins field names with " + " to create summation expression
expression = " + ".join(value_fields)

# Calculate the Value_Sum field
arcpy.management.CalculateField(out_union, new_field, expression, "PYTHON3")
```

III. Reviewing the Script

3. 3. Quantify Proximity to Objects

3.1 Dissolve Degradation Sources

3.2 Generate Buffers for Degradation Sources

3.3 Assign Buffer Values

3.4 Generate Donut Buffers

3.5 Merge Donut Buffers

III. Reviewing the Script

3.3. Quantify Proximity to Objects

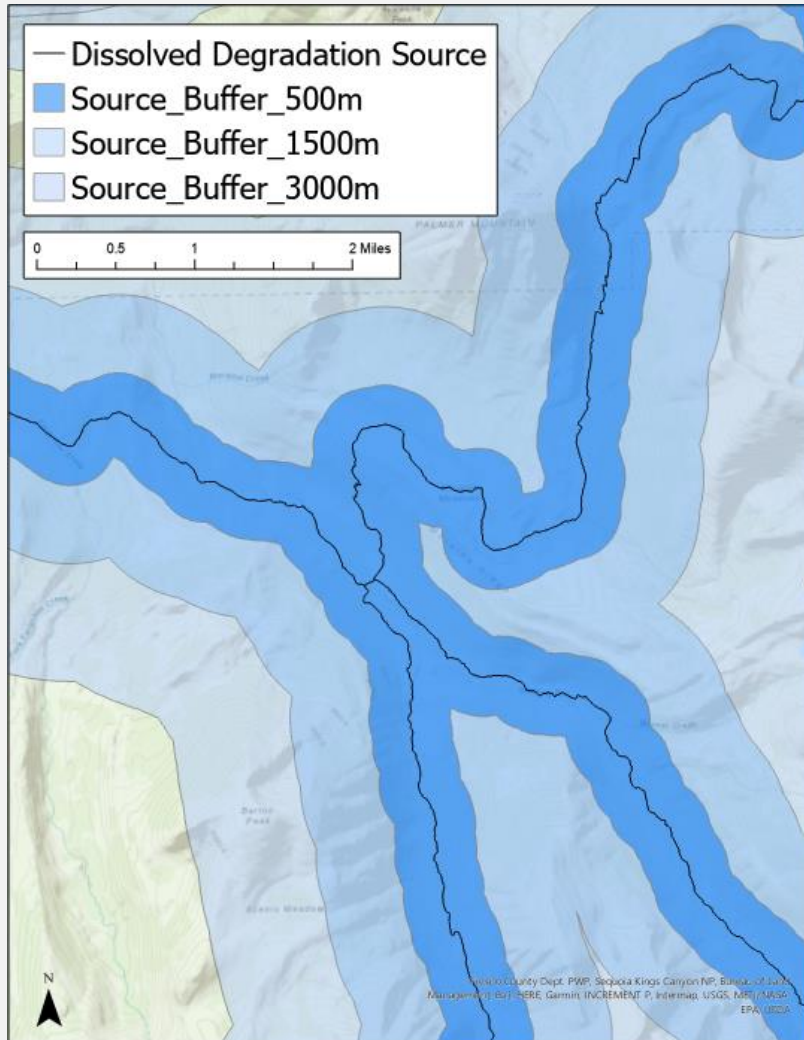
3.1 Dissolve Degradation Sources

3.2 Generate Buffers for Degradation Sources

3.3 Assign Buffer Values

3.4 Generate Donut Buffers

3.5 Merge Donut Buffers



Here, trails are buffered according to the script's default settings: one buffer at 500m, one at 1500m, and one at 3000m

3.1 Dissolve Degradation Sources

The purpose of this step is to merge all contiguous segments of the degradation source feature class into single polygons or lines. This ensures that each contiguous section is treated as a single feature during buffering and eliminates false overlaps.

```
# Dissolve the degradation source feature class
dissolved_degradation = rf"{output_gdb}\Dissolved_Degradation"
arcpy.management.Dissolve(
    in_features=source_of_degradation,
    out_feature_class=dissolved_degradation
)
```

3.2 Generate Buffers for Degradation Sources

This section dynamically creates buffer zones around degradation sources at user-defined distances. These buffers represent zones of ecological impact, which will be used in later analyses to assess overlaps with critical habitat.

Steps:

1. initialize a list for collecting buffer layers
2. iterate through user-defined buffer distances
3. generate buffers
4. collect calculated buffers in a list

```
# generate initial buffers
buffered_layers = []
for distance in buffer_distances:
    buffer_output = rf"{output_gdb}\Source_Buffer_{distance}m"
    arcpy.analysis.Buffer(
        in_features=dissolved_degradation,
        out_feature_class=buffer_output,
        buffer_distance_or_field=f"{distance} Meters",
        dissolve_option="NONE"
    )
    buffered_layers.append(buffer_output)
```

III. Reviewing the Script

3.3. Quantify Proximity to Objects

3.1 Dissolve Degradation Sources

3.2 Generate Buffers for Degradation Sources

3.3 Assign Buffer Values

3.4 Generate Donut Buffers

3.5 Merge Donut Buffers

500m Buffer, Value = 5

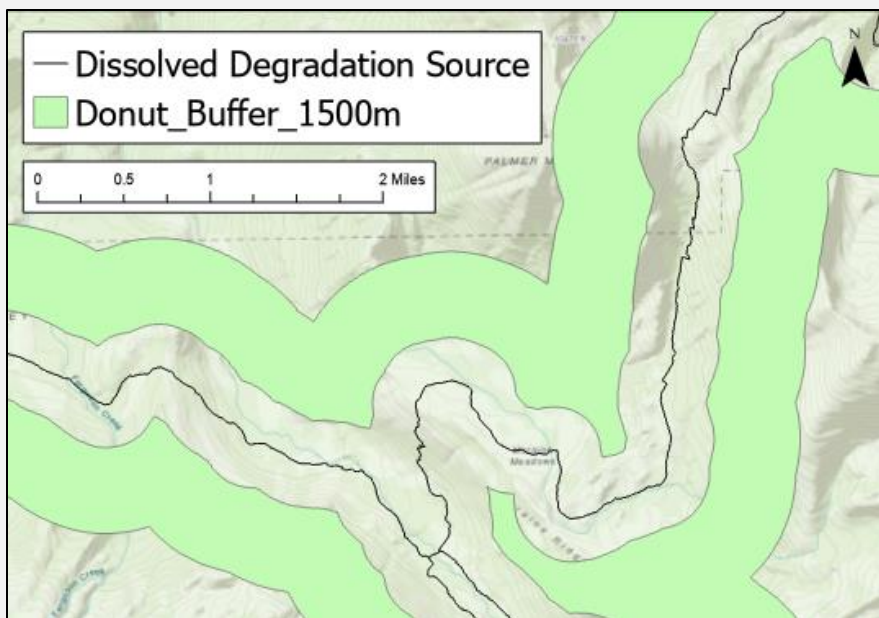
	OBJECTID_1 *	Shape *	BUFF_DIST	ORIG_FID	Shape_Length	Shape_Area	BufferValue
1	1	Polygon	500	1	10946890.199731	6039635203.809902	5

1500m Buffer, Value = 3

	OBJECTID_1 *	Shape *	BUFF_DIST	ORIG_FID	Shape_Length	Shape_Area	BufferValue
1	1	Polygon	1500	1	8092258.362435	15461028178.648899	3

3000m Buffer, Value = 1

	OBJECTID_1 *	Shape *	BUFF_DIST	ORIG_FID	Shape_Length	Shape_Area	BufferValue
1	1	Polygon	3000	1	4754075.281414	24893172422.201347	1



3.3 Assign Buffer Values

Assign restoration potential scores to each buffer based on proximity to the degradation source, using the user-defined `buffer_values` dictionary.

Steps:

1. iterate through zipped values from the buffered layers list and buffer distances dictionary
2. add & calculate the new buffer value field

```
# assign values to buffers
for buffer_output, distance in zip(buffered_layers, buffer_distances):
    arcpy.management.AddField(buffer_output, "BufferValue", "SHORT")
    buffer_value = buffer_values[distance]
    arcpy.management.CalculateField(buffer_output, "BufferValue", buffer_value, "PYTHON3")
```

3.4 Generate Donut Buffers

Generates non-overlapping buffer zones by subtracting inner buffers from outer buffers, isolating specific distance ranges.

1. iterate over buffer layers from the largest to the smallest.
2. use the Erase tool to subtract the inner buffer from the outer buffer
3. include the innermost buffer as it is

```
donut_buffers = []
for i in range(len(buffer_distances) - 1, 0, -1):
    outer_buffer = buffered_layers[i]
    inner_buffer = buffered_layers[i - 1]
    donut_output = rf"{output_gdb}\Donut_Buffer_{buffer_distances[i]}m"
    arcpy.analysis.Erase(
        in_features=outer_buffer,
        erase_features=inner_buffer,
        out_feature_class=donut_output
    )
    donut_buffers.append(donut_output)

donut_buffers.append(buffered_layers[0])
```

Here, only the 1500m donut buffer is displayed to show that it is distinct from the other buffer donuts.

III. Reviewing the Script

3.3. Quantify Proximity to Objects

3.1 Dissolve Degradation Sources

3.2 Generate Buffers for Degradation Sources

3.3 Assign Buffer Values

3.4 Generate Donut Buffers

3.5 Merge Donut Buffers

3.5 Merge Donut Buffers

Combines all donut buffers into a single feature class.

```
# Path for merged output
merged_buffers = rf"{output_gdb}\Merged_Donut_Buffers"

# Combine all donut buffers
arcpy.management.Merge(
    inputs=donut_buffers,
    output=merged_buffers
)
```

Merged donut buffers. Note that merged features retain their unique BufferValues.

	OBJECTID_1 *	Shape *	BUFF_DIST	ORIG_FID	BufferValue	Shape_Length	Shape_Area
1	1	Polygon	3000	1	1	12238897.493396	8354986396.136292
2	2	Polygon	3000	2	1	7356635.236474	3861770581.287629
3	3	Polygon	1500	1	3	19054771.430445	8788904482.496092
4	4	Polygon	1500	2	3	9304687.577316	3390699740.174237
5	5	Polygon	500	1	5	10946890.199731	6039635203.809902
6	6	Polygon	500	2	5	4250657.579091	1897865408.360729

III. Reviewing the Script

4.Value Intersection Zones

4.1 Intersect Merged Donut Buffers with Habitat Overlap

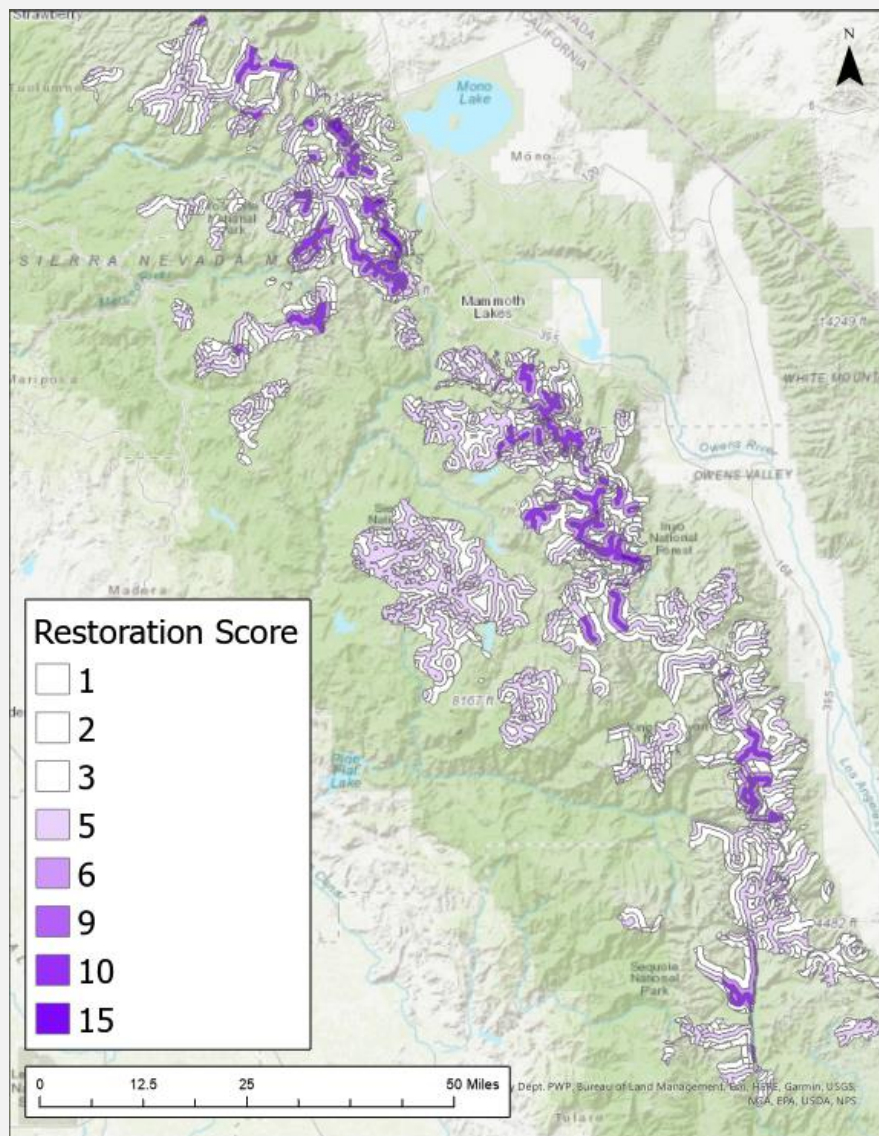
4.2 Assign Each Polygon a Restoration Value Score

III. Reviewing the Script

4.Value Intersection Zones

4.1 Intersect Merged Donut Buffers with Habitat Overlap

4.2 Assign Each Polygon a Restoration Value Score



4.1 Intersect Merged Donut Buffers with Critical Habitat Zones

Identifies areas for restoration potential.

```
# Path for intersected output
restoration_with_habitat = rf"{output_gdb}\Restoration_With_Habitat"

# Intersect merged buffers with habitat zones
arcpy.analysis.Intersect(
    in_features=[out_union, merged_buffers], # Habitat zones and merged buffers
    out_feature_class=restoration_with_habitat,
    join_attributes="ALL" # Retain all attributes
)
```

4.2 Assign Each Polygon a Restoration Value Score

Calculates the restoration potential for each feature by multiplying the buffer values with the habitat overlap values

```
# Add a field for restoration potential score
arcpy.management.AddField(restoration_with_habitat, "RestorationScore", "FLOAT")

# Calculate restoration potential score
restoration_expression = "!BufferValue! * !Overlap_Value!"
arcpy.management.CalculateField(
    in_table=restoration_with_habitat,
    field="RestorationScore",
    expression=restoration_expression,
    expression_type="PYTHON3"
)
```

III. Reviewing the Script

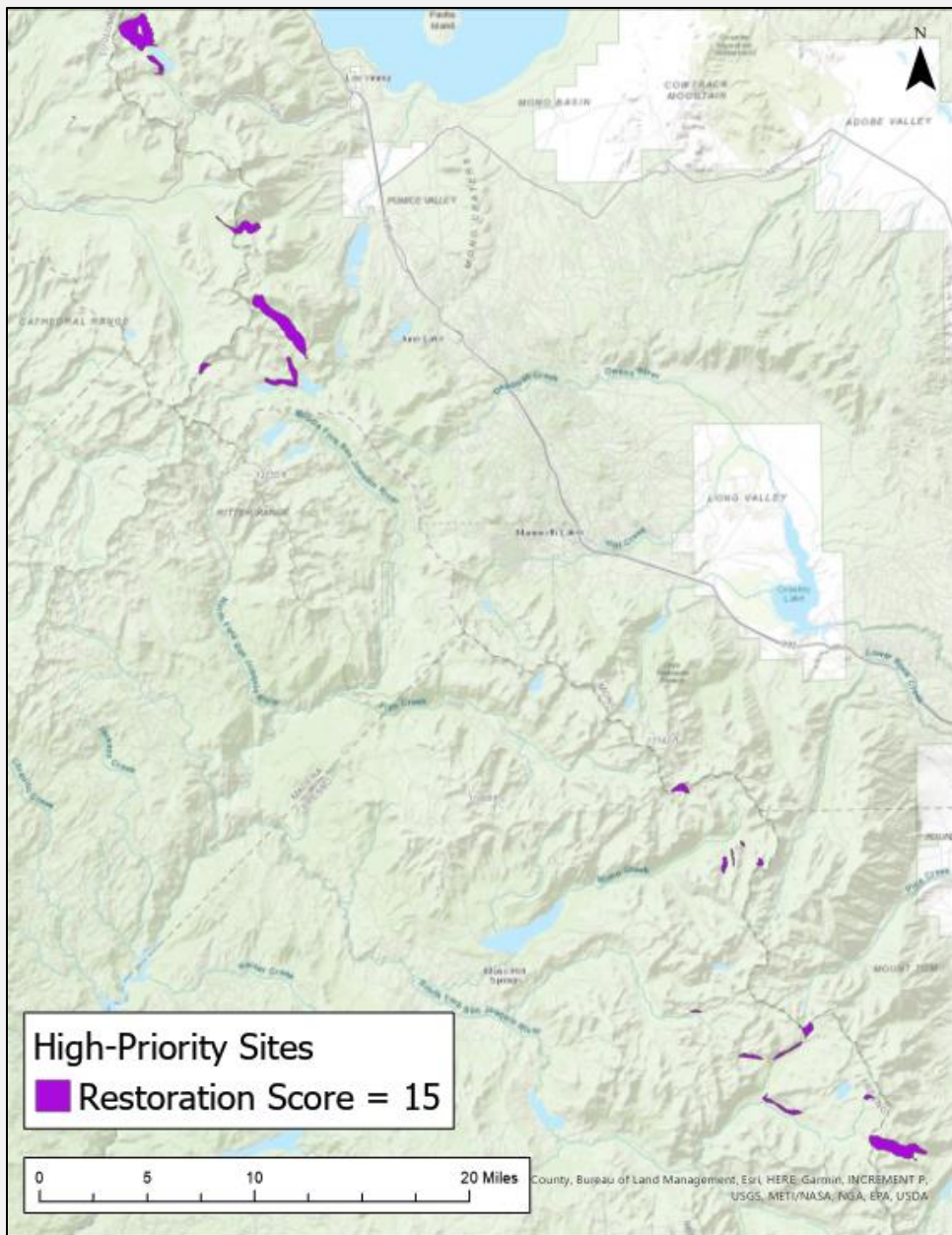
5. Isolate Sites with Maximum Value

5.1 Isolate Polygons with the maximum Restoration Value Score

III. Reviewing the Script

5. Isolate Sites with Maximum Value

5.1 Isolate Polygons with the maximum Restoration Value Score



5.1 Isolate Polygons with the Maximum Restoration Value Score

These polygons constitute the final output of this tool: the highest-priority restoration sites in an area of interest.

```
# Make a feature layer for the restoration data
arcpy.management.MakeFeatureLayer(
    in_features=restoration_with_habitat,
    out_layer="RestorationLayer"
)

# Select polygons with the maximum RestorationScore
arcpy.management.SelectLayerByAttribute(
    in_layer_or_view="RestorationLayer",
    selection_type="NEW_SELECTION",
    where_clause="RestorationScore = (SELECT MAX(RestorationScore) FROM Restoration_With_Habitat)"
)

# Define the output feature class for high-priority sites
highest_priority_sites = rf"{output_gdb}\Highest_Priority_Sites"

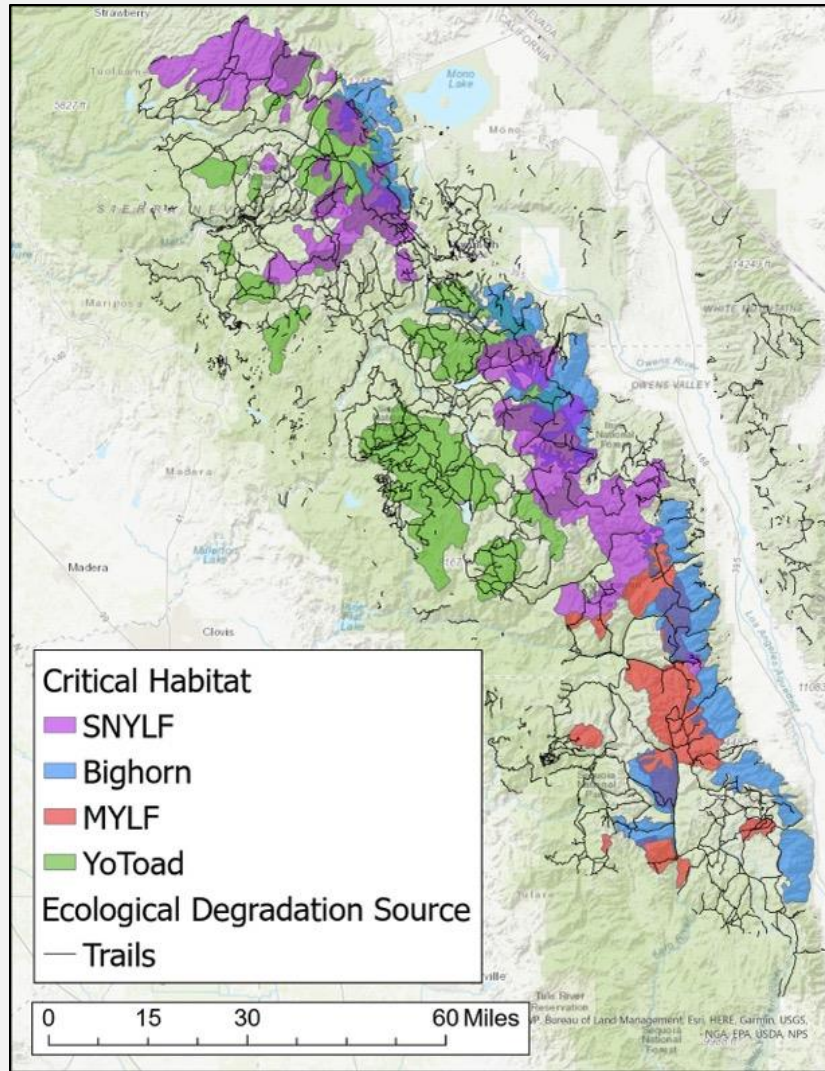
# Copy the selected features
arcpy.management.CopyFeatures(
    in_features="RestorationLayer",
    out_feature_class=highest_priority_sites
)

# Define the output for the dissolved feature class
dissolved_priority_sites = rf"{output_gdb}\Dissolved_Highest_Priority_Sites"

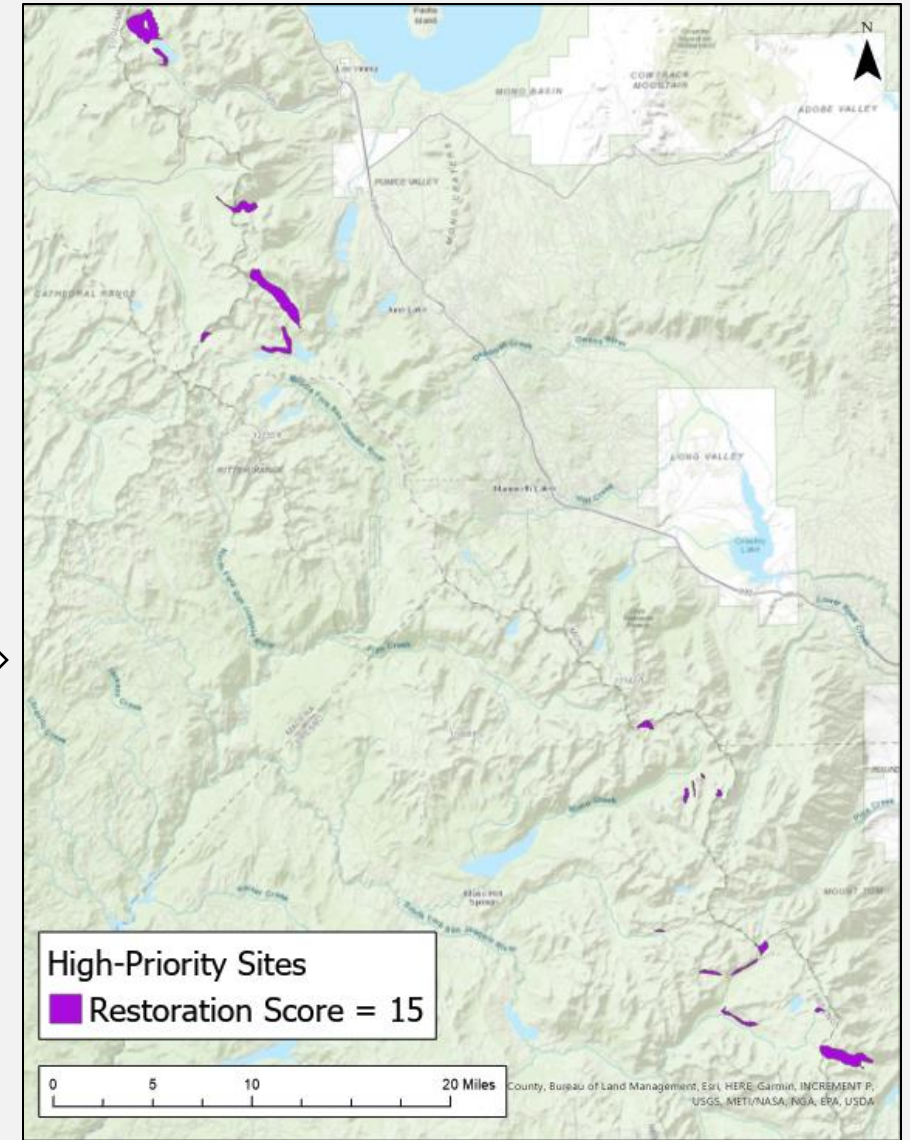
# Dissolve contiguous polygons by RestorationScore
arcpy.management.Dissolve(
    in_features=highest_priority_sites,
    out_feature_class=dissolved_priority_sites,
    dissolve_field="RestorationScore",
    multi_part="SINGLE_PART"
```

INPUT & OUTPUT MAPS

These maps serve as a visual reminder of how The Prioritizer transforms inputs into the final output feature class.



The Prioritizer



IV. TESTING GENERALIZABILITY - THE SCENARIO:

Suppose that I am biologist working in the Los Padres National Forest instead of the Sierra Nevada. There are no Sierra Nevada bighorn sheep or mountain yellow-legged frogs here, but we do have our own local ESA-listed species. I am particularly concerned with the California condor, California red-legged frog, Arroyo toad, and vernal pool fairy shrimp. Additionally, I'm not so worried about hiking trails as a source of ecological degradation, but I am quite concerned about impacts from motor vehicles on backcountry Forest Service roads.

A deadline is approaching to apply for a grant funding ecological restoration projects to benefit ESA-listed species, but I'm not sure how to effectively site projects in my proposal. Wouldn't it be great if there were a generalized software tool available to help identify high-priority ecological restoration sites?

As it happens, a colleague at Yale School of the Environment sent me a python script called The Prioritizer to do exactly that. Let's see if it can help me out...

IV. TESTING GENERALIZABILITY

WHAT'S NEW?

As previously stated, this software tool is designed to allow users to easily input custom data and parameters to adapt it for their specific needs. In this generalizability test, I updated the geodatabase paths, specified new species habitat feature classes and their common names, selected a different source of ecological degradation, and defined new buffer distances and values. The tool ran smoothly and completed the analysis without any errors.

Just the user-defined inputs.

```
# user-defined base directory and geodatabase paths
base_dir = r"C:\Modeling Geographical Objects\Data_Automation\Data_Automation_Report" # base
input_gdb = rf"{base_dir}\Generalize_Input.gdb" # new input GDB for the new project
output_gdb = rf"{base_dir}\Generalize_Output.gdb" # new output GDB for the new project

# List of input species feature classes
species_list = [
    rf"{input_gdb}\ArroyoToad_LP", # new species critical habitat fc 1
    rf"{input_gdb}\Condor_LP", # new species critical habitat fc 2
    rf"{input_gdb}\CRLF_LP", # new species critical habitat fc 3
    rf"{input_gdb}\FairyShrimp_LP" # new species critical habitat fc 4
]

# corresponding common names for each species
common_names = ["ArroyoToad", "CaliCondor", "RedLeggedFrog", "FairyShrimp"] # correspond to new species

# ecological degradation source feature class
source_of_degradation = rf"{input_gdb}\ForestRoads" # feature class of roads instead of trails

# buffer distances (meters)
buffer_distances = [1000, 3000, 5000] # different buffer distances

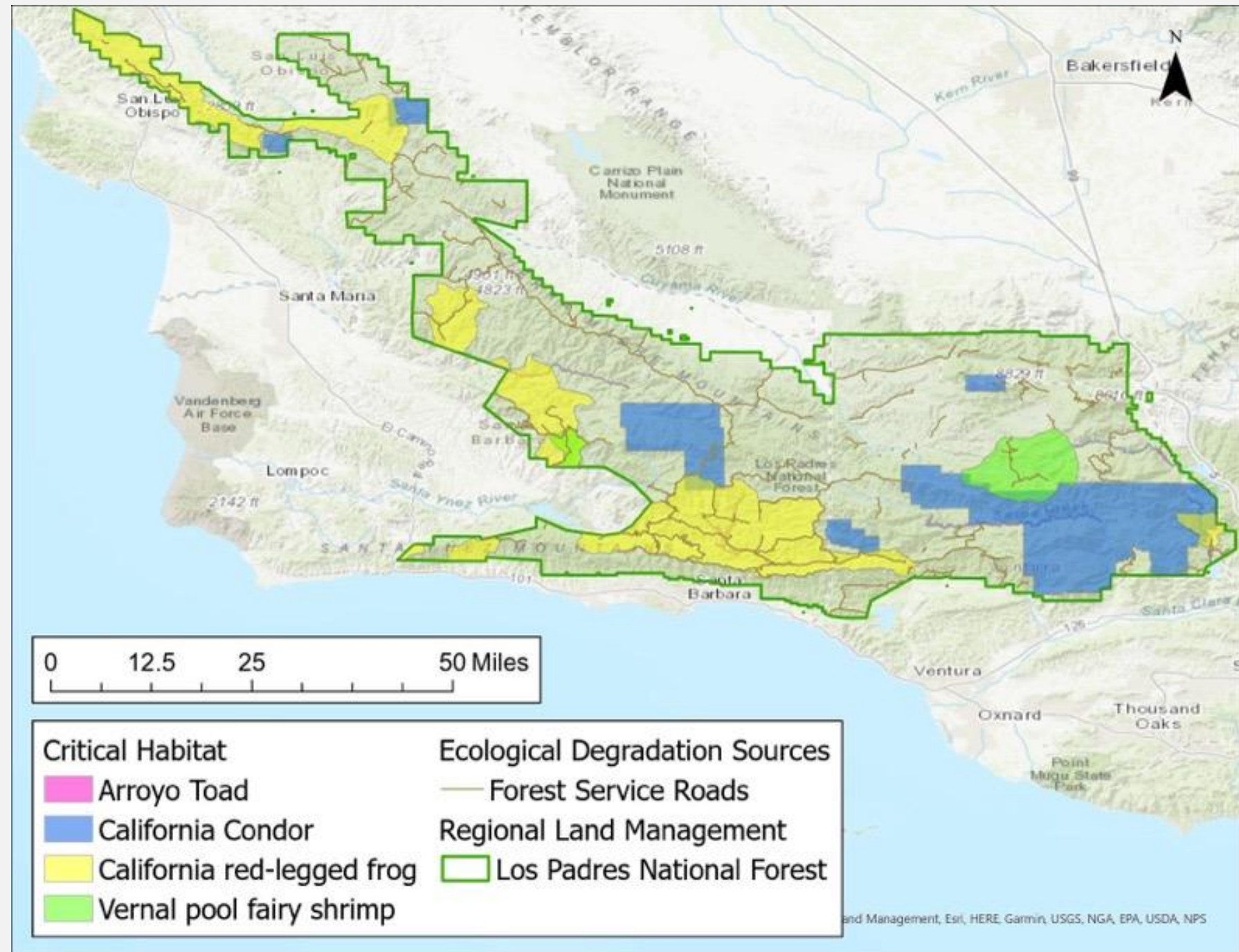
# buffer values
buffer_values = {1000: 10, 3000: 5, 5000: 2} # different buffer values
```

IV. TESTING GENERALIZABILITY

INPUTS MAP

Our hypothetical biologist has input their custom data and parameters and is ready to run the The Prioritizer tool.

This map is a visual representation of the hypothetical biologist's input data.



IV. TESTING GENERALIZABILITY

OUTPUT MAP & CONCLUSION

This map shows the final output feature class representing high-priority ecological restoration sites in the Los Padres National Forest, as determined by The Prioritizer tool.

The Prioritizer identified these sites based on their proximity to Forest Service roads and the number of ESA-listed species utilizing them as critical habitat.

In conclusion, the hypothetical biologist was able to take a geospatial software tool originally developed for use in the Sierra Nevada and apply it for their own purposes in a different region, with different data and parameters. This shows that The Prioritizer has been sufficiently generalized beyond its initial scope.

